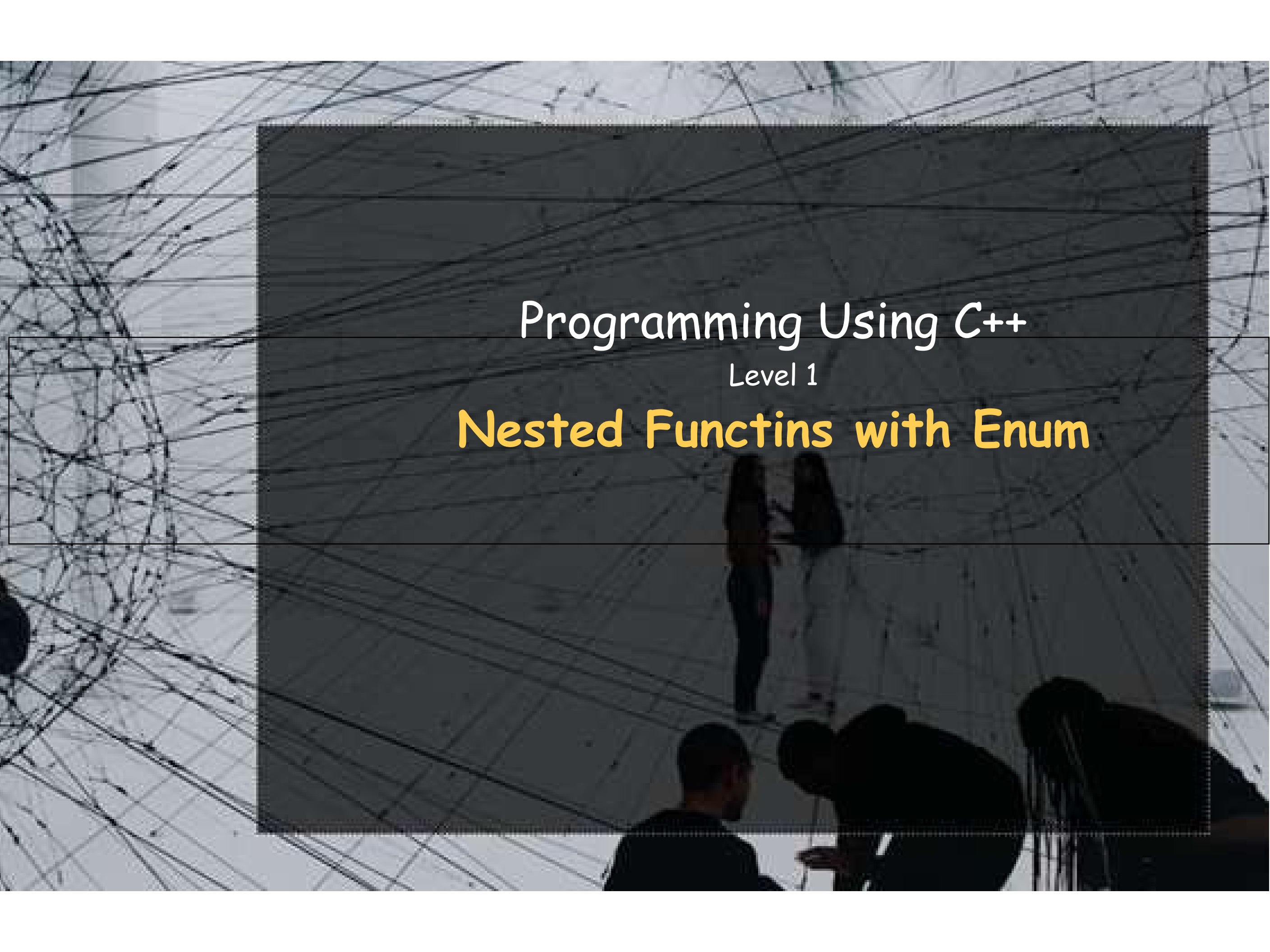


Programming Using C++ Level 1

Nested Functions with Enum



Programming Using C++

Level 1

Nested Functions with Enum

🔍 Definition

- ◆ Nested functions in this context means using multiple functions together, where the result of one function is used as input for another function.
- ◆ An enum can represent a fixed set of operation types.
- ◆ This program combines functions, an enum, and a switch statement to build a simple calculator.

Explanation

◆ The program uses four operation types:

 ADD → +

 SUBTRACT → -

 MULTIPLY → *

 DIVIDE → /

◆ ReadNumber() reads a number from the user.

◆ ShowOperationsMenu() displays the available operations.

◆ ReadOperation() reads the operation symbol and converts it into the enOperation enum.

◆ Calculate() receives the two numbers and the selected operation, then uses switch to perform the correct calculation.

Steps in main()

1. Define the enum

- ◆ Create `enOperation` and assign each operation its corresponding symbol.

2. Read the first number

- ◆ Use `ReadNumber()` to receive the first number.

3. Read the second number

- ◆ Use `ReadNumber()` to receive the second number.

4. Display the operations menu

- ◆ Call `ShowOperationsMenu()` to show the available operations.

5. Read the operation

- ◆ Call `ReadOperation()` to read the selected operation.

6. Perform the calculation

- ◆ Pass both numbers and the operation to `Calculate()`.

7. Select the operation

- ◆ Use `switch` to determine which calculation should be performed.

8. Display the result

- ◆ Print the value returned by `Calculate()`.

💡 Example

◆ If the user enters:

First number: 10

Second number: 5

Operation: *

◆ The operation is converted to:

enOperation::MULTIPLY

◆ Calculate() performs:

$10 * 5$

◆ The result is:

Result = 50

➡ Result

◆ Example results:

$10 + 5 \rightarrow 15$

$10 - 5 \rightarrow 5$

$10 * 5 \rightarrow 50$

$10 / 5 \rightarrow 2$

◆ If the user tries to divide by zero, the program displays:

Error: Division by zero!

Note

- ◆ The enum values are assigned directly to the operation characters:

ADD = '+'

SUBTRACT = '-'

MULTIPLY = '*'

DIVIDE = '/'

- ◆ enOperation(op) converts the character entered by the user into the enum type.
- ◆ Calculate() returns a double because the result of division may contain decimal values.
- ◆ The division condition checks num2 != 0 before performing division.

⊘ Warning

- ◆ Division by zero is not allowed.
- ◆ The program handles this case by displaying an error message and returning 0.



Comparison

▨ Using Multiple Functions

- ✓ Each function has a specific responsibility.
- ✓ Makes the program easier to understand and maintain.
- ✓ Functions can be reused independently.

▤ One Large Function

- ✓ All operations are placed in one block of code.
- ✓ Becomes harder to read as the program grows.
- ✓ Makes individual tasks harder to reuse.

✦✦ Summary

- ✓ Multiple functions can work together to divide a program into clear and reusable tasks.
- ✓ Enums can represent different operation types and work with switch statements.
- ✓ Combining functions, enums, and switch creates a clean structure for a calculator program.

Thank you 😊